Kodgranskning och SAST-verktyg

Lektionsuppgift: granska koden i lek-4.js och kör ett SAST-verktyg

Namn: Sima Soolind

Kurs: Säker applikationslivscykel

Datum: 6 november 2026

1. Uppgiften

Jag fick en liten Node.js-app med Express och MySQL. Appen har en inloggningssida (/login) och en meddelandesida (/messages). Uppgiften hade tre delar:

1. **Manuell granskning:** leta efter SQL injection och XSS i koden.
2. **SAST-verktyg:** köra koden i ett verktyg och se vad det varnar för.
3. **Fixar:** laga sårbarheterna (frivillig del).

SAST betyder Static Application Security Testing. Ett SAST-verktyg läser koden och letar efter sårbarheter utan att köra programmet.

2. Manuell granskning

Jag läste koden själv. Jag följde texten som användaren skriver in, från req.body och req.query, till platsen där den används. Jag skrev en kommentar vid varje farlig rad.

Utdrag ur lek-4.js med mina kommentarer:

```
//sårbarhet nr 4: hårdkodade hemligheter , lösenordet syns i koden.
const db = mysql.createConnection({ ... password: 'root' ... });

//sårbarhet nr 1: SQL-injektion , saknar paramateserade frågor.
const query = "SELECT * FROM users WHERE username = '" + username + ...

//sårbarhet nr 5: felet från databasen visas för användaren.
return res.send('Fel inträffade vid databasfråga: ' + err);

//sårbarhet nr 2: xss , användarnamnet visas utan output encoding.
res.send('Välkommen <b>' + username + '</b>, du är inloggad!');

// xss attack i url:en genom att lägga script i en länk
res.send('<h1>Dina meddelanden</h1><p>' + msg + '</p>');
```

Det här hittade jag:

Nr	Sårbarhet	Var	Varför det är farligt
1	SQL injection	/login, SQL-frågan	Användarnamn och lösenord sätts rakt in i SQL-strängen. En angripare kan logga in utan lösenord.
2	XSS	/login, välkomsttexten	Användarnamnet skrivs rakt in i HTML. Ett skript i namnet körs i webbläsaren.
3	XSS (reflected)	/messages	Texten i länken skrivs rakt in i HTML. En angripare kan skicka en länk med ett skript.
4	Hårdkodade hemligheter	Databasanslutningen	Lösenordet till databasen står i koden. Alla som ser koden kan logga in.
5	Felmeddelandet visas	/login, vid databasfel	Hela felet från databasen visas för användaren. Det ger angriparen information.

Jag såg också att koden inte kontrollerar tillåtna värden och längd i användarnamnet (ingen validering).

3. SAST-verktyg 1: Semgrep

Jag körde Semgrep i Docker på filen lek-4.js. Semgrep körde 104 regler.

```
docker run --rm -v ~/scan:/src -w /src semgrep/semgrep semgrep scan \
  --config p/javascript --config p/xss --config p/sql-injection \
  --config p/secrets --metrics=off lek-4.js
```

Utdrag ur resultatet i terminalen:

```
4 Code Findings
  javascript.express.security.injection.raw-html-format
    User data flows into ... manually-constructed HTML ... (XSS)
    53| res.send('Välkommen <b>' + username + '</b>, du är inloggad!');
  javascript.express.security.audit.xss.direct-response-write
    53| res.send('Välkommen <b>' + username + '</b>, du är inloggad!');
  javascript.express.security.injection.raw-html-format
    62| res.send('<h1>Dina meddelanden</h1><p>' + msg + '</p>');
  javascript.express.security.audit.xss.direct-response-write
    62| res.send('<h1>Dina meddelanden</h1><p>' + msg + '</p>');

Ran 104 rules on 1 file: 4 findings.
```

Resultat: 4 varningar. Alla gällde XSS på två rader (välkomsttexten och /messages). Varje rad fick två varningar, eftersom två olika regler slog till.

Semgrep missade: SQL injection, det hårdkodade lösenordet och felmeddelandet.

4. SAST-verktyg 2: Bearer

Jag körde Bearer i Docker på samma fil. Bearer körde 87 regler och hittade 8 sårbarheter.

```
docker run --rm -v ~/scan:/tmp/scan bearer/bearer:latest-amd64 scan /tmp/scan
```

Sammanfattning från terminalen:

```
87 checks, 8 findings
CRITICAL: 2 (CWE-798, CWE-89)
HIGH: 4 (CWE-79)
MEDIUM: 2 (CWE-693)
LOW: 0
```

Allvar	CWE	Vad Bearer hittade
CRITICAL	CWE-89	SQL injection
CRITICAL	CWE-798	Hårdkodat lösenord
HIGH	CWE-79	XSS i välkomsttexten och i /messages (4 varningar)
MEDIUM	CWE-693	Helmet saknas: servern skickar inga säkerhetsrubriker till webbläsaren
MEDIUM	CWE-693	Fingerprinting: servern visar att den använder Express

CWE är en officiell lista över typer av sårbarheter. Varje typ har ett nummer.

Bearer missade: felmeddelandet som visas för användaren (min nr 5).

5. Jämförelse: jag, Semgrep och Bearer

Sårbarhet	Jag	Semgrep	Bearer
SQL injection	Ja	Nej	Ja
XSS i välkomsttexten	Ja	Ja	Ja
XSS i /messages	Ja	Ja	Ja
Hårdkodat lösenord	Ja	Nej	Ja
Felmeddelandet visas	Ja	Nej	Nej
Helmet saknas	Nej	Nej	Ja
Fingerprinting	Nej	Nej	Ja

Ingen hittade allt. Jag hittade 5 av 7. Bearer hittade 6 av 7. Semgrep hittade 2 av 7.

6. Fixar

Jag lagade koden i en ny fil, lek-4-fixad.js. Min granskning i lek-4.js är kvar som den var.

Nr	Sårbarhet	Fix
1	SQL injection	Parametriserad fråga: ? som platshållare och värdena i en separat lista
2–3	XSS	escapeHtml() gör om specialtecken som < och > till vanlig text
4	Hårdkodat lösenord	Lösenordet ligger i filen .env och hämtas med process.env
5	Felmeddelandet	Användaren får ett allmänt fel. Felkoden skrivs bara i serverns terminal.
6–7	Helmet och fingerprinting	app.use(helmet()) och app.disable('x-powered-by')
8	Ingen validering	Kontroll av typ och längd innan frågan når databasen

Exempel: SQL injection före och efter

Före:

```
const query = "SELECT * FROM users WHERE username = '" + username
  + "' AND password = '" + password + "'";
```

Efter:

```
const query = 'SELECT * FROM users WHERE username = ? AND password = ?';
db.query(query, [username, password], (err, rows) => {
```

Databasen får frågan och värdena separat. Därför behandlas värdena alltid som text och aldrig som SQL-kod.

Kontroll: körde verktygen igen

Jag körde Bearer igen på den lagade filen.

```
87 checks, 5 findings
CRITICAL: 0
HIGH: 4 (CWE-79)
MEDIUM: 0
LOW: 1 (CWE-532)
```

Allvar	Före	Efter
CRITICAL	2	0
HIGH	4	4 (falsk positiv, se nedan)
MEDIUM	2	0
LOW	0	1

XSS-varningarna finns kvar trots att XSS är lagad. Verktygen känner inte igen paketet escape-html. De tror därför att texten fortfarande är farlig. Det kallas en **falsk positiv**: verktyget varnar för något som inte är ett problem. Jag kontrollerade koden själv och vet att texten escapas.

7. Slutsats: människa och verktyg behövs tillsammans

Det viktigaste jag har lärt mig är att **jag inte kan lita blint på ett verktyg**. Men jag kan inte heller lita bara på mig själv.

Verktyget räcker inte ensamt

- Semgrep missade SQL injection, som är den allvarligaste sårbarheten i koden.
- Inget verktyg hittade att felmeddelandet visar för mycket. Det kräver att en människa förstår vad informationen kan användas till.
- Verktygen varnade för XSS i den lagade koden, fast den var säker (falsk positiv). Om jag hade litat blint hade jag lagt tid på fel sak.
- Ett verktyg som inte varnar betyder inte att koden är säker.

Jag räcker inte ensam

- Bearer hittade att Helmet saknas och att servern avslöjar Express. Det missade jag.
- Verktyg är snabba, tröttnar inte och kör samma regler varje gång.

Därför

- Jag använder **flera verktyg**, eftersom de har olika regler och hittar olika saker.
- Jag **granskar själv** och kontrollerar varje varning: är den äkta eller en falsk positiv?
- Det är **jag som utvecklare som är ansvarig** för koden, inte verktyget.

Det här kallas **defense in depth**: flera skyddslager, så att ett fel i ett lager fångas av ett annat.

8. Koppling till mina projekt

I **ParishHub** kommer det att finnas inloggning och medlemsdata. Tro är känsliga personuppgifter enligt GDPR artikel 9. Därför gäller samma risker som i lek-4.js:

- Jag använder **Prisma**, som skapar parametriserade frågor automatiskt (skydd mot SQL injection).
- Jag validerar indata med **Zod**.
- Hemligheter ligger i **.env** på servern, aldrig i koden.
- Lösenord hashas med **bcrypt**.
- Jag lägger in Semgrep eller Bearer i **CI**, så att koden granskas automatiskt varje gång jag pushar. Jag granskar ändå själv.